

EE708: Fundamentals of Data Science and Machine Intelligence

Rajesh M Hegde
Dept. of EE IIT Kanpur



Module 6B Backpropagation in Neural Networks, Training DNNs

Backpropagation in Neural Networks

- It is a method used to train artificial neural networks.
- The goal is to minimize the cost/ loss function iteratively, by adjusting weights and biases.
- The algorithm consists of three simple steps:

Present one or more training examples to the neural network.

Compare the output of the neural network to the desired value.

Adjust the weights to make the output closer to the desired value.

Backpropagation in Neural Networks

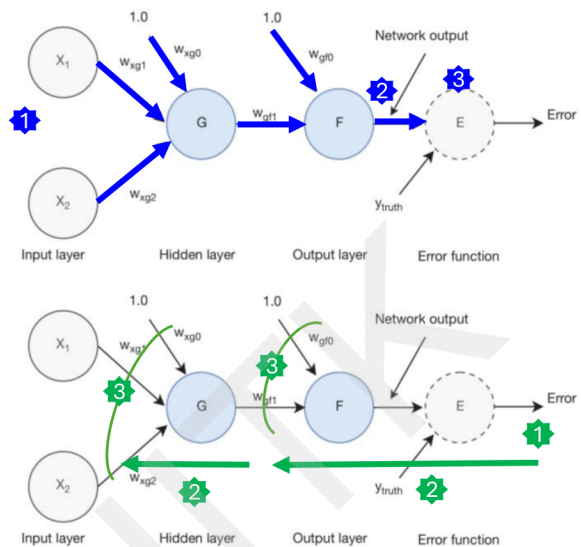
- *Backpropagation* simplifies and efficiently computes partial derivatives with respect to weights in neural networks.
- It consists of forward pass followed by backward pass.

Forward pass

- 1 Presents a training example
- 2 Computes network output
- 3 Compares it to the ground truth.

Backward pass

- 1 Computes the error/loss
- 2 Computes partial derivatives (error gradient)
- 3 Adjusts weights using gradient descent.



EE708

Module 6

3

Partial Derivatives for Multilevel/Complex Networks

- To apply partial derivatives, **the function that it is applied to is to be differentiable** to compute the gradient.
- In partial derivatives for Perceptron, the activation (sign) function was ignored and partial derivative of z was done.
- Computing of partial derivatives for a multilevel network with multiple neurons per layer is complex.
- This cannot be used when working with a multilayer network where the outputs from the activation functions in one layer are used as inputs to the next layer.
- It is **nontrivial due to the dependency on activation functions across layers**.

EE708

Module 6

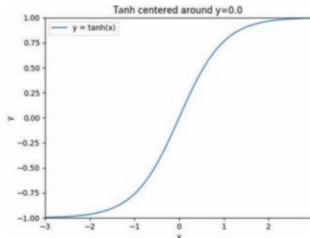
4

Activation Functions

Hyperbolic tangent

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

- Symmetric around 0 on both axis

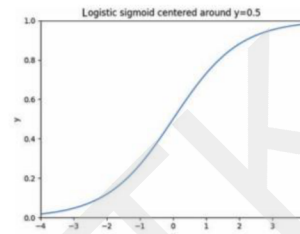


$$\tanh'(x) = 1 - \tanh^2(x)$$

Logistic sigmoid

$$S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}$$

- Symmetric around 0 on the x-axis and around 0.5 on the y-axis.



$$S'(x) = S(x)(1 - S(x))$$

Derivatives can be computed from the function values even if the x-values are not available.

EE708

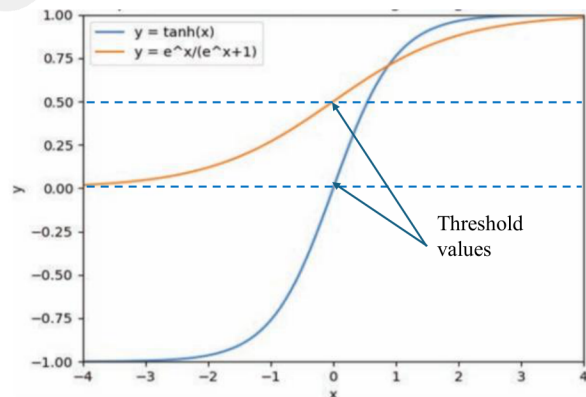
Module 6

5

Derivatives of Activation Functions

- Activation functions include many types, with some being S-shaped (e.g., tanh and logistic sigmoid).
- Recommendation:
 - Use tanh for hidden layers
 - Logistic sigmoid for the output layer

	<i>sign</i> <i>/tanh</i>	Logistic sigmoid	Symmetric digital
Max. output	1	1	1
Min. output	-1	0	0
Threshold	0	0.5	0.5



Comparing tanh and logistic sigmoid functions.

EE708

Module 6

6

Backpropagation : Derivatives of Composite Functions

- **Composite Function:** Let $f(x)$ and $g(x)$ be two functions.

- The function combined as a composite function as

$$h(x) = f(g(x)) \approx f \circ g(x) \approx f \circ g$$

- **Chain rule** (also known as *Leibniz's notation*) states that

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial g} \frac{\partial g}{\partial x}$$

- The derivative of a composition of functions:

$$\frac{\partial h(x)}{\partial x} = f'(g(x))g'(x) = (f' \circ g)g'$$

- In **neural networks**, functions have multiple variables.

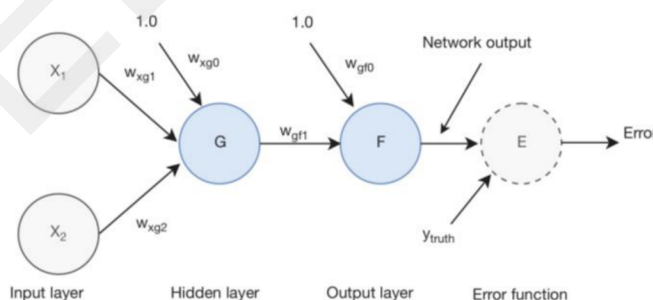
EE708

Module 6

7

Using Backpropagation to Learn the Weights

- The neural network has two neurons **G** and **F**
 - **G** has **three** adjustable weights and **F** has **two** adjustable weights
- **Objective:** develop an algorithm to find weights for the desired functionality.
- **E** in the figure represents a function, not part of the network.
- **E** : Evaluate how correct or incorrect the network's output is, aiding the learning process.



Simple two-layer network used to explain backpropagation. The last unit (dashed) is not a part of the network but represents the error function that compares the output to ground truth.

EE708

8

Using Backpropagation to Compute the Weights

- The input and the weights are defined as

$$\mathbf{x}_G = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \quad \mathbf{x}_F = \begin{bmatrix} 1 \\ y_G \end{bmatrix} \quad \mathbf{w}_G = \begin{bmatrix} w_{xg0} \\ w_{xg1} \\ w_{xg2} \end{bmatrix} \quad \mathbf{w}_F = \begin{bmatrix} w_{gf0} \\ w_{gf1} \end{bmatrix}$$

- At neuron G :**

$$z_G = \mathbf{x}_G^T \mathbf{w}_G = w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2$$

- Output of neuron G :

$$y_G = \tanh(z_G) = \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2)$$

- At neuron F :**

- $z_F = \mathbf{x}_F^T \mathbf{w}_F = w_{gf0} + w_{gf1} \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2)$

- Output of the network is**

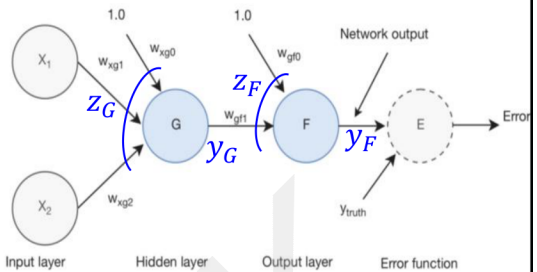
$$\hat{y} = y_F = \mathbf{S}(z_F) = \mathbf{S}(w_{gf0} + w_{gf1} \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2))$$

$\mathbf{S}$ is the Logistic sigmoid activation function

EE708

Module 6

9



Using Backpropagation to Compute the Weights : Error

- Let y be the desired signal. The error is

$$Error = y - \hat{y}$$

- The gradient of the *error function* is calculated instead of the *Error*:

- The error function is defined as $e = \frac{(y - \hat{y})^2}{2}$

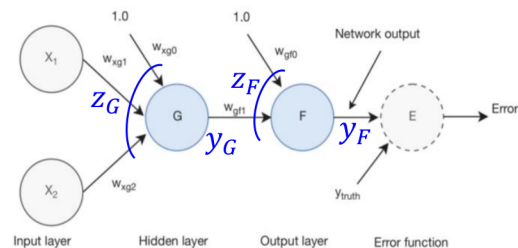
- The mean square error (cost function) of the overall network

$$MSE = \frac{1}{n} \sum_{i=1}^n e = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- The gradient descent is used to minimize the MSE by adjusting the weights.

- Gradient descent** is used to minimize function by weight update as: $\mathbf{w} - \eta \nabla e$

- ∇e is gradient of error function with respect to weights $\mathbf{w}$



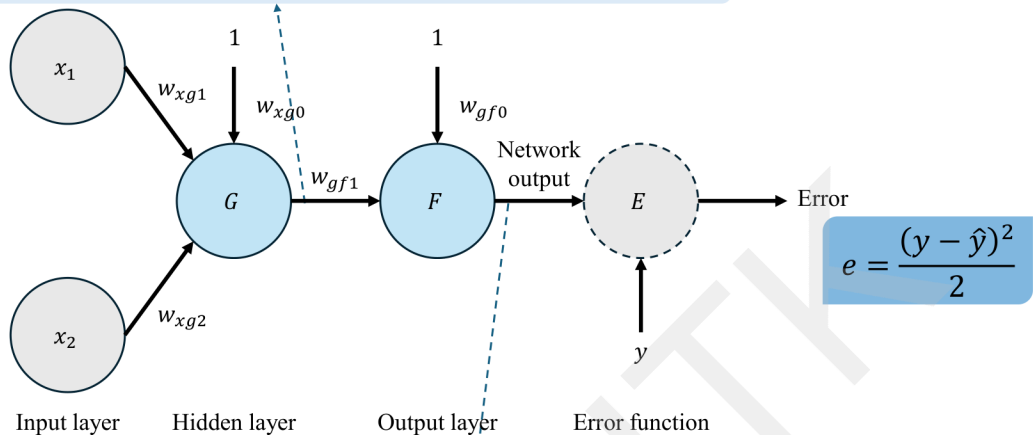
EE708

Module 6

10

Using Backpropagation to Compute the Gradient : Error Function

$$y_G = \tanh(\mathbf{x}^T \mathbf{w}_G) = \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2)$$



$$\hat{y} = S(z_F) = S(w_{gf0} + w_{gf1} \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2))$$

EE708

Module 6

12

Using Backpropagation (at Neuron F) to Compute the Gradient : Error Function

- Overall, the **error function** that to minimize can be written as a composite function:

$$e = \frac{1}{2} \left(y - S(\underbrace{w_{gf0} + w_{gf1} \tanh(\underbrace{w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2}_{z_G})}_{y_G}) \right)^2$$

$$\underbrace{\hspace{10em}}_{z_F}$$

$$\hat{y}$$

$$\triangleq e(\hat{y}(z_F(\mathbf{w}_F, y_G(z_G(\mathbf{w}_G))))))$$

- $\mathbf{x}_G$ and y are not variables but treated as constants for a given training example.

- Partial derivative w.r.t $\mathbf{w}_F = [w_{gf0}, w_{gf1}]^T$:

$$\nabla_{\mathbf{w}_F} e = \frac{\partial e}{\partial \mathbf{w}_F} = \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial \mathbf{w}_F} = \begin{bmatrix} \frac{\partial e}{\partial w_{gf0}} \\ \frac{\partial e}{\partial w_{gf1}} \end{bmatrix} = \begin{bmatrix} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf0}} \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf1}} \end{bmatrix}$$

EE708

Module 6

13

Using Backpropagation (at F) to Compute the Gradient

$$\nabla_{\mathbf{w}_F} e = \frac{\partial e}{\partial \mathbf{w}_F} = \left[\begin{array}{ccc} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf0}} & & \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf1}} & & \end{array} \right] \left. \vphantom{\frac{\partial e}{\partial \mathbf{w}_F}} \right\} \begin{array}{l} \text{To compute} \\ \text{we need} \\ e, \hat{y}, z_F \end{array}$$

$$\begin{aligned} e &= \frac{(y - \hat{y})^2}{2} \\ \hat{y} &= S(z_F) \\ z_F &= \mathbf{x}_F \mathbf{w}_F = w_{gf0} + w_{gf1} y_G \end{aligned}$$

$$\nabla_{\mathbf{w}_F} e = \left[\begin{array}{ccc} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf0}} & & \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial w_{gf1}} & & \end{array} \right] = \left[\begin{array}{c} -(y - \hat{y}) S'(z_F) \\ -(y - \hat{y}) S'(z_F) y_G \end{array} \right]$$

2 gradients

$$\begin{aligned} \bullet \frac{\partial e}{\partial \hat{y}} &= -(y - \hat{y}) \\ \bullet \frac{\partial \hat{y}}{\partial z_F} &= S'(z_F) \\ \bullet \frac{\partial z_F}{\partial w_{gf0}} &= 1 \\ \bullet \frac{\partial z_F}{\partial w_{gf1}} &= y_G \end{aligned}$$

• Three key observations:

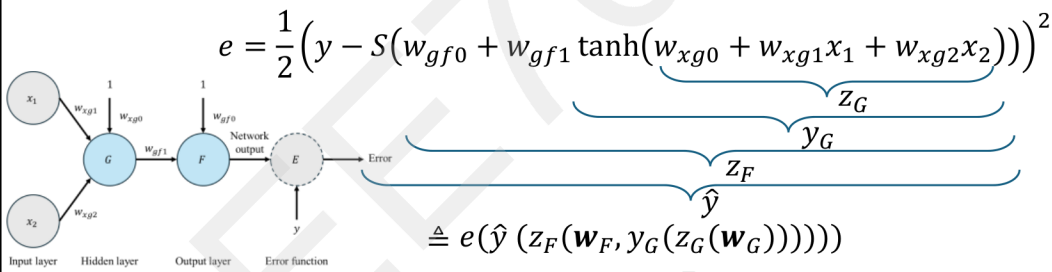
1. $y, \hat{y}, z_F, y_G$ are known from **forward pass**.
2. **Conscious choice** of activation function makes derivative of S possible.
3. The derivative of S depends on S itself.

EE708

Module 6

14

Using Backpropagation (at G) to Compute the Gradient



• Partial derivative w.r.t $\mathbf{w}_G = [w_{xg0}, w_{xg1}, w_{xg2}]^T$:

$$\nabla_{\mathbf{w}_G} e = \frac{\partial e}{\partial \mathbf{w}_G} = \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial \mathbf{w}_G} = \left[\begin{array}{c} \frac{\partial e}{\partial w_{xg0}} \\ \frac{\partial e}{\partial w_{xg1}} \\ \frac{\partial e}{\partial w_{xg2}} \end{array} \right] = \left[\begin{array}{ccccc} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg0}} & & & & \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg1}} & & & & \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg2}} & & & & \end{array} \right]$$

EE708

Module 6

15

Using Backpropagation to (at G) Compute the Gradient

$$\nabla_{W_G} e = \left[\begin{array}{c} \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg0}} \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg1}} \\ \frac{\partial e}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_F} \frac{\partial z_F}{\partial y_G} \frac{\partial y_G}{\partial z_G} \frac{\partial z_G}{\partial w_{xg2}} \end{array} \right]$$

To compute
we need
 $e, \hat{y}, z_F,$
 y_G, z_G

$$\begin{aligned} e &= \frac{(y - \hat{y})^2}{2} \\ \hat{y} &= S(z_F) \\ z_F &= \mathbf{x}_F \mathbf{w}_F = w_{gf0} + w_{gf1} y_G \\ y_G &= \tanh(z_G) \\ z_G &= w_{xg0} + w_{xg1} x_1 + w_{xg2} x_2 \end{aligned}$$

$$\nabla_{W_G} e = \left[\begin{array}{c} -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) \\ -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) x_1 \\ -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) x_2 \end{array} \right]$$

3 gradients

$$\begin{aligned} \bullet \frac{\partial e}{\partial \hat{y}} &= -(y - \hat{y}) & \bullet \frac{\partial z_G}{\partial w_{xg0}} &= 1 \\ \bullet \frac{\partial \hat{y}}{\partial z_F} &= S'(z_F) & \bullet \frac{\partial z_G}{\partial w_{xg1}} &= x_1 \\ \bullet \frac{\partial z_F}{\partial y_G} &= w_{gf1} & \bullet \frac{\partial z_G}{\partial w_{xg2}} &= x_2 \\ \bullet \frac{\partial y_G}{\partial z_G} &= \tanh'(z_G) \end{aligned}$$

EE708

Module 6

16

Using Backpropagation (at both F and G) to Compute the Gradient

- All **five derivatives** that the gradient consists of:

$$\begin{aligned} \frac{\partial e}{\partial w_{gf0}} &= -(y - \hat{y}) S'(z_F) \\ \frac{\partial e}{\partial w_{gf1}} &= -(y - \hat{y}) S'(z_F) y_G \\ \frac{\partial e}{\partial w_{xg0}} &= -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) \\ \frac{\partial e}{\partial w_{xg1}} &= -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) x_1 \\ \frac{\partial e}{\partial w_{xg2}} &= -(y - \hat{y}) S'(z_F) w_{gf1} \tanh'(z_G) x_2 \end{aligned}$$

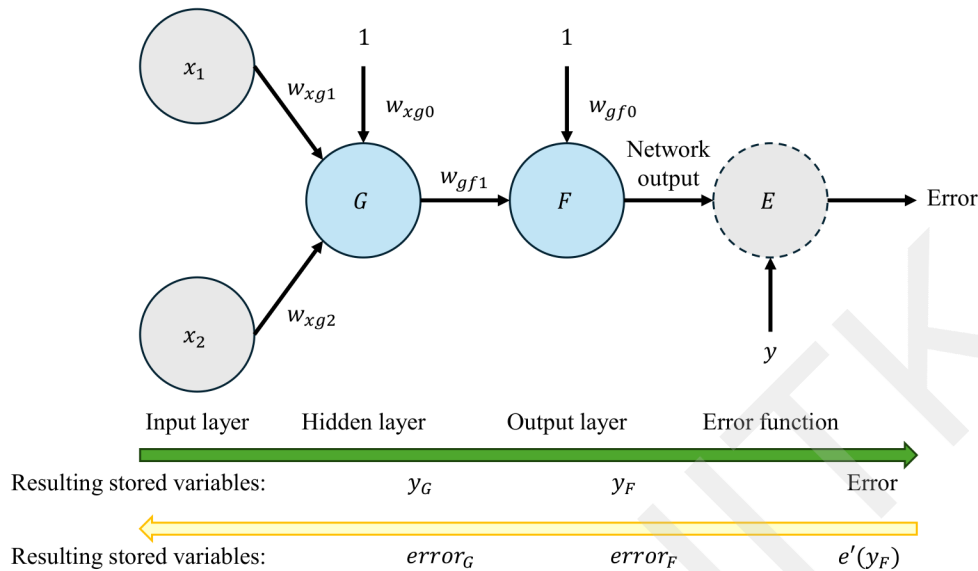
- The derivative of tanh, like the derivative of S, is also simple to compute.

EE708

Module 6

17

Using Backpropagation to Compute the Gradient



EE708

Module 6

18

Using Backpropagation to Compute the Gradient

Network learning algorithm based on gradient descent using backpropagation

- Forward pass:** Compute and store activation function output (y) for each neuron and finally the error.
- Backward pass:**
 - Compute the derivatives of the error function.
 - Compute (Backpropagate) the error** for each neuron by multiplying the error from the subsequent neuron (that it feeds) by the weight of that neuron and then multiply by the derivative of its activation function.

Derivatives are computed from the stored output of the activation function.

- Update weights:** $w_{\text{new}} = w_{\text{old}} - (\eta \times \text{input} \times \text{error})$

where **input** is the input value to that weight

error is the error term for the neuron the weight belongs

EE708

Module 6

19

Backpropagation: Adjusting Weights

- Weight adjustment in gradient descent depends on several components:
 - Overall error: A big error should lead to a big adjustment.
 - All weights and derivatives (on the path from the weight in question to the error at the end of the network): If one or more weights or derivatives on this path will suppress the effect of this weight change, then it is not helpful to change it.
 - Input to the weight: If the input to the weight is small, then adjusting the weight will not have much of an effect.
- Overall, these observations make intuitive sense for how to identify which weights should get significant adjustments.

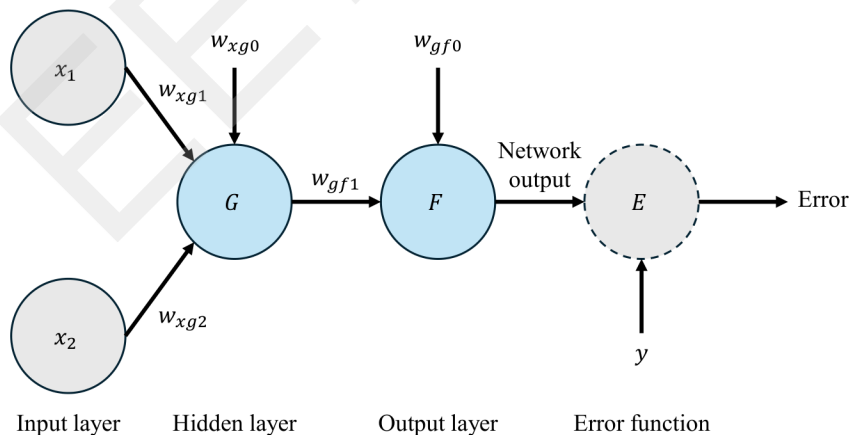
EE708

Module 6

21

Backpropagation: Numerical Example

- Initial weights: $w_{xg0} = 0.3$, $w_{xg1} = 0.6$, $w_{xg2} = -0.1$, $w_{gf0} = -0.2$, $w_{gf1} = 0.5$
- Training example : $x_1 = -0.9$, $x_2 = 0.1$, $y = 5$ • Learning rate: $\eta = 0.1$



EE708

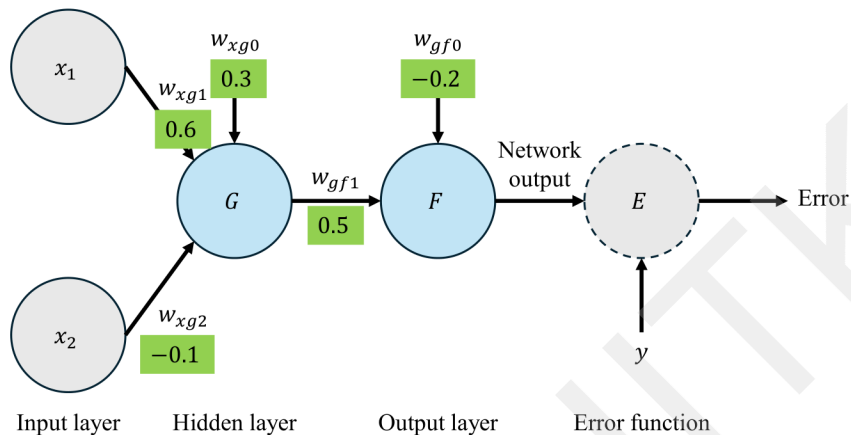
Module 6

22

Numerical Example : Weights for Iteration 1

Initial weights for 1st iteration

$$w_{xg0} = 0.3, \quad w_{xg1} = 0.6, \quad w_{xg2} = -0.1, \quad w_{gf0} = -0.2, \quad w_{gf1} = 0.5$$



EE708

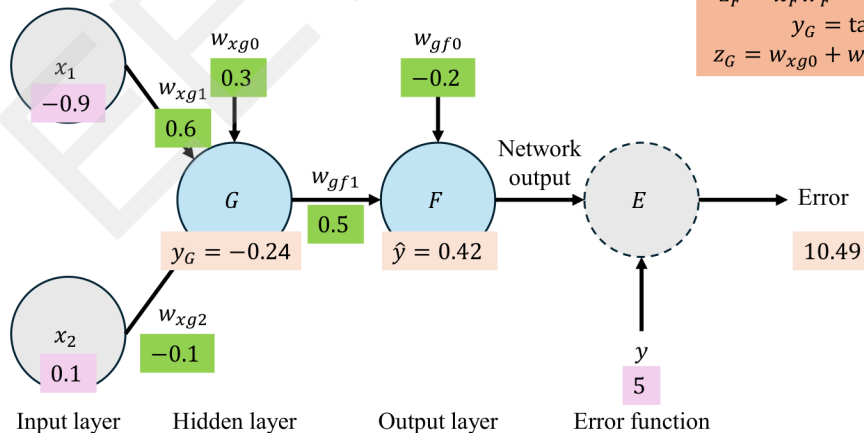
Module 6

23

Numerical Example : First Forward Pass – Output and Error Compute

1st Forward Pass

$$x_1 = -0.9, \quad x_2 = 0.1, \quad y = 5$$



$$e = \frac{(y - \hat{y})^2}{2}$$

$$\hat{y} = S(z_F)$$

$$z_F = \mathbf{x}_F \mathbf{w}_F = w_{gf0} + w_{gf1} y_G$$

$$y_G = \tanh(z_G)$$

$$z_G = w_{xg0} + w_{xg1} x_1 + w_{xg2} x_2$$

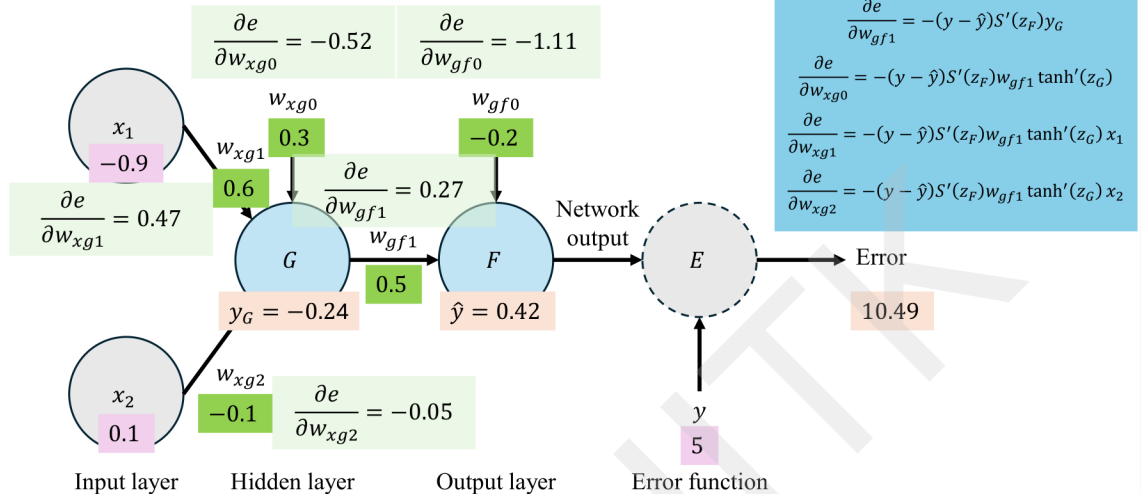
EE708

Module 6

24

Numerical Example : First Backward Pass – Gradient Compute

1st Backward Pass



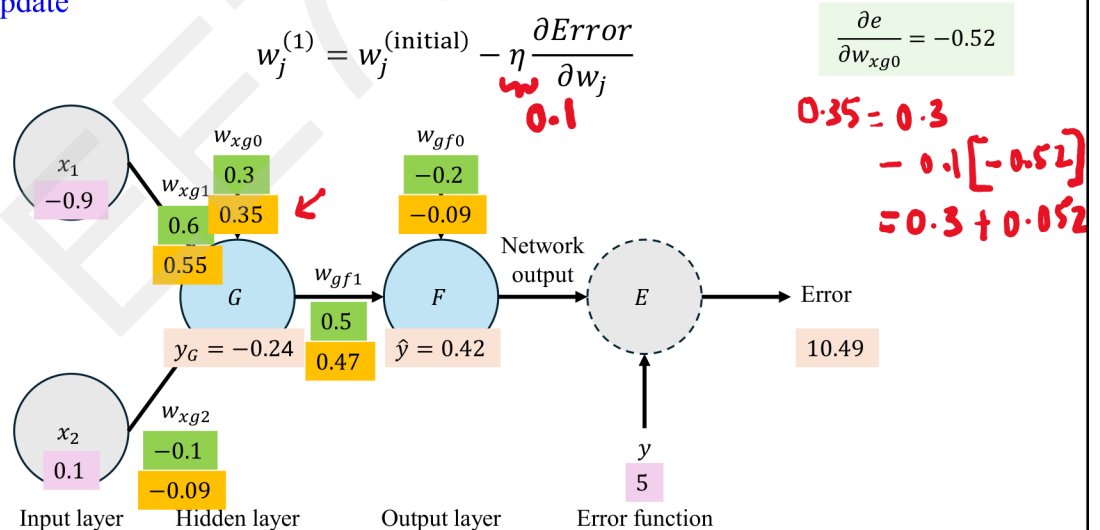
EE708

Module 6

25

Numerical Example : First Backward Pass – Weights Update

1st Weight update



EE708

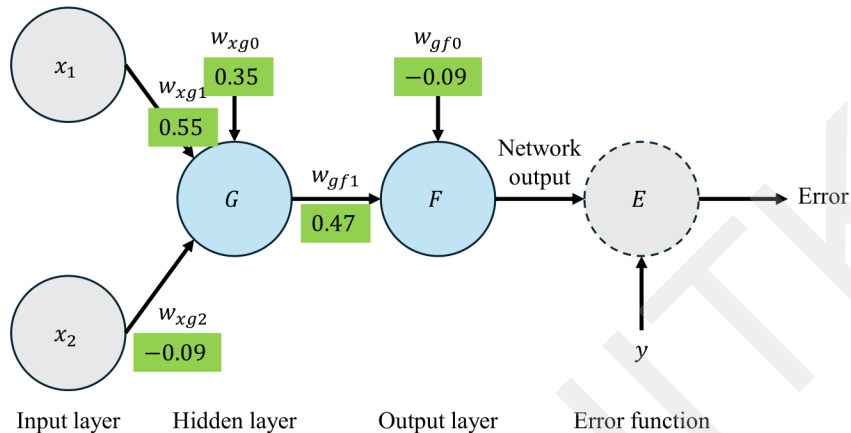
Module 6

27

Numerical Example : Start Second Iteration with Updated Weights

Initial weights for 2nd iteration

$$w_{xg0} = 0.35, w_{xg1} = 0.55, w_{xg2} = -0.09, w_{gf0} = -0.09, w_{gf1} = 0.47$$



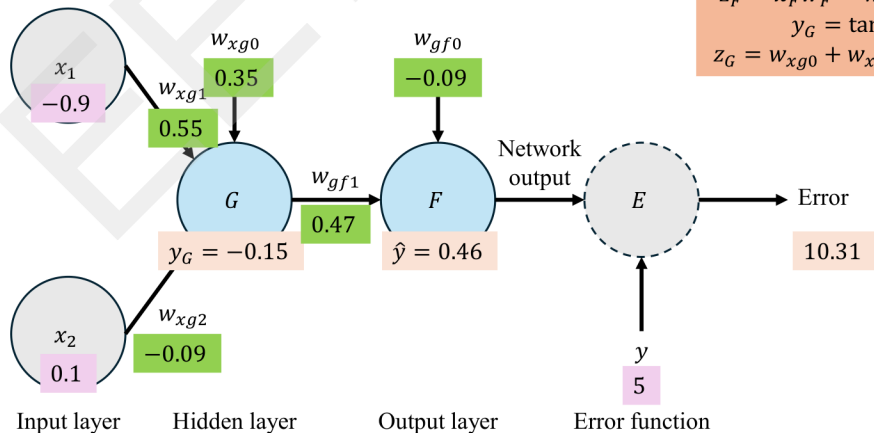
EE708

Module 6

28

Numerical Example : Second Forward Pass

2nd Forward Pass



$$e = \frac{(y - \hat{y})^2}{2}$$

$$\hat{y} = S(z_F)$$

$$z_F = \mathbf{x}_F \mathbf{w}_F = w_{gf0} + w_{gf1} y_G$$

$$y_G = \tanh(z_G)$$

$$z_G = w_{xg0} + w_{xg1} x_1 + w_{xg2} x_2$$

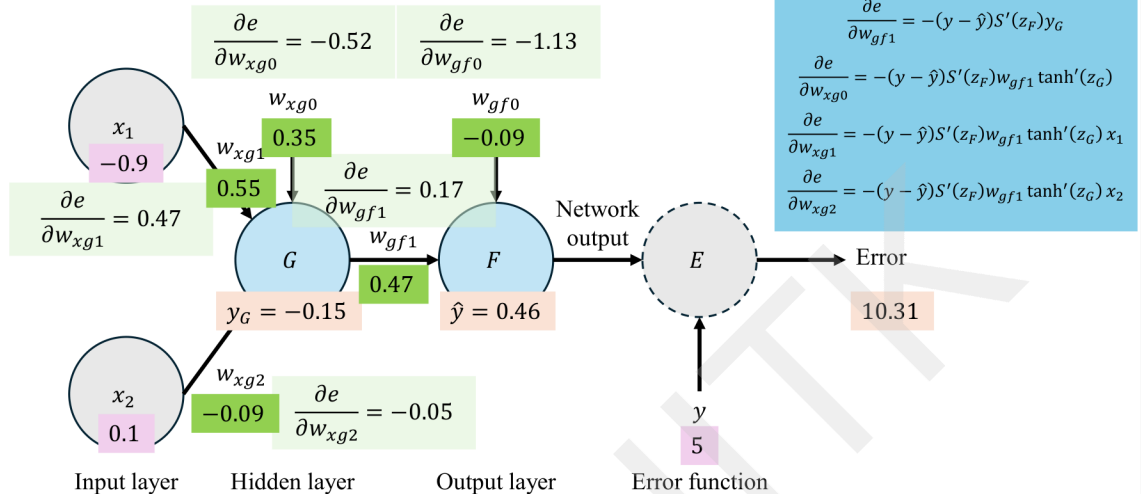
EE708

Module 6

29

Numerical Example : Second Backward Pass

2nd Backward Pass



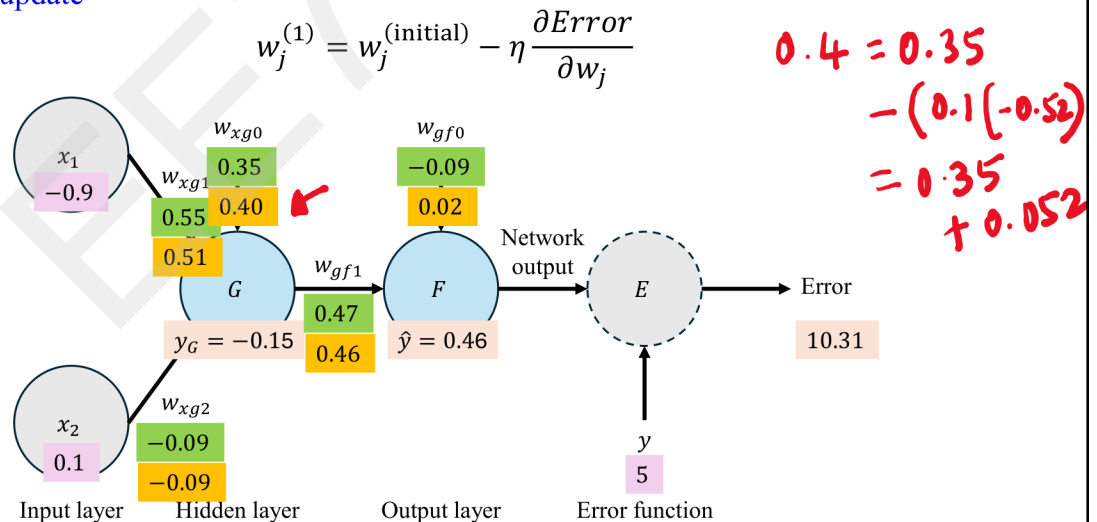
EE708

Module 6

30

Numerical Example : Weights Update on Second Pass

2nd Weight update



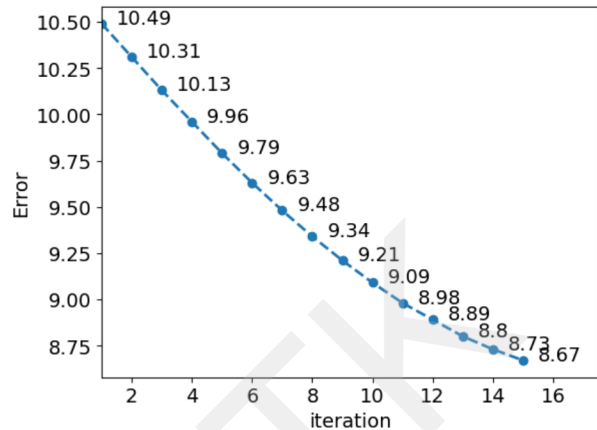
EE708

Module 6

31

Numerical Example : Analysis

iteration	w_{xg0}	w_{xg1}	w_{xg2}	w_{gf0}	w_{gf1}	Error
1	0.3	0.6	-0.1	-0.2	0.5	10.49
2	0.35	0.55	-0.09	-0.09	0.47	10.31
3	0.40	0.51	-0.09	0.02	0.46	10.13
4	0.46	0.46	-0.08	0.14	0.45	9.96
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
13	0.83	0.12	-0.05	0.97	0.71	8.80
14	0.86	0.10	-0.04	1.04	0.75	8.73
15	0.89	0.07	-0.04	1.10	0.79	8.67



- With iterations, error decreases slowly.
- What would be the effect on the error if η is increased to 0.2 or 0.5?
- How would error change if weights are updated with new samples at each iteration?

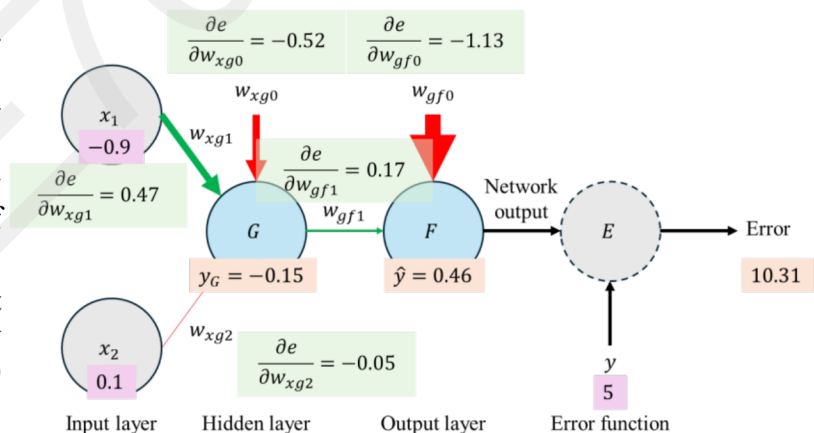
EE708

Module 6

32

Numerical Example : Illustration and General Inference

- A green arrow represents a positive weight adjustment, and red represents a negative adjustment.
- The width of the arrow indicates the magnitude of the adjustment.
- Actual weights are not shown in the figure; only the adjustment value (delta) is shown.



Network annotated with numbers computed during forward and backward passes.

EE708

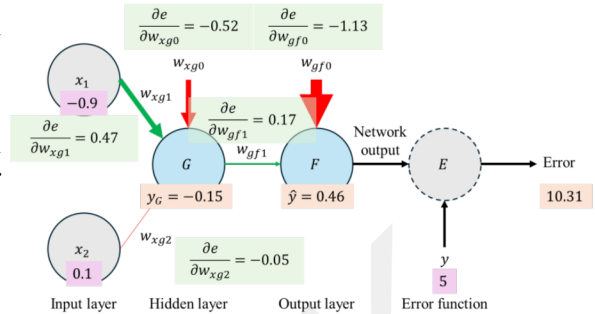
Module 6

35

Numerical Example : General Inference

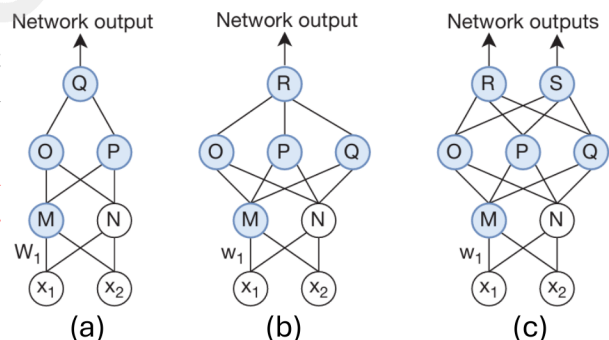
Intuitions drawn from the magnitude and direction of the weight adjustments:

- Considering neuron G, the weights for the bias term and the x_1 input are adjusted by an order of magnitude more than the weight for x_2 (-0.52 and 0.47 for the bias and x_1 weights vs. -0.05 for the x_2 weight).
- This makes sense because the magnitude of the bias input and x_1 is greater than the magnitude for x_2 , and thus, these two weights are more significant levers.**
- Output is less than the desired output, so we want to increase the network output.
- This property, together with the sign of the input values that feed each weight, will determine the direction in which the weight is adjusted.
- For example, the bias weight is increased, whereas the weight corresponding to input x_1 is decreased because the x_1 input is a negative value.



Backpropagation with Multiple Neurons per Layer

- The error term for a neuron contains weighted errors from all subsequent neurons instead of just a single weighted error term.
- Lets See an Example of a **complex network**
- Example: computation of the **error term for neuron M** for the network requires
 - (a) the weighted errors from O and P.
 - (b) the weighted errors from O, P, and Q.
 - (c) the weighted errors from O, P, and Q.
- For the **network (c)**
 - Two **output neurons (R and S)**.
 - Use the **weighted errors** for R and S when computing the **error terms** for O, P, and Q.

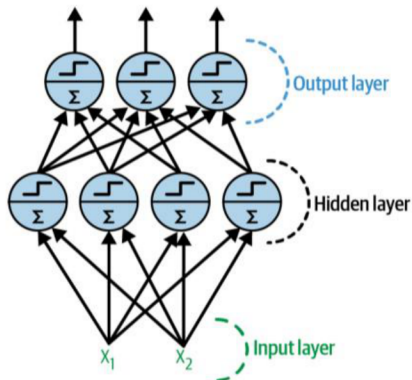


More complex networks.

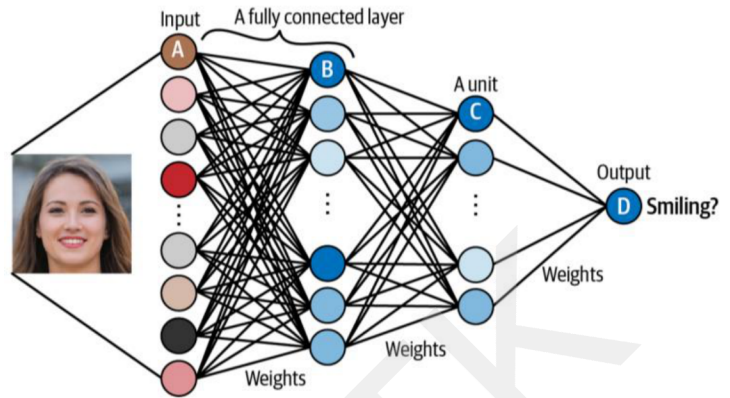
Inputs are at the bottom, and layers are stacked vertically instead of horizontally.

$$e = \frac{1}{2} ((y_R - \hat{y}_R)^2 + (y_S - \hat{y}_S)^2)$$

Example of Multilayer Perceptron



The architecture of a multilayer perceptron with two inputs, one hidden layer of four neurons, and three output neurons



An example of a multilayer perceptron that predicts if a face is smiling

EE708

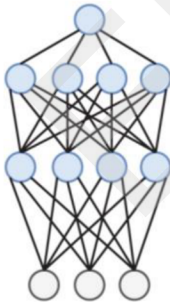
Module 6

39

Different Network Architectures

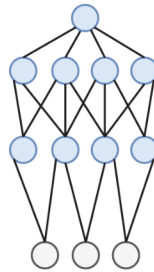
Fully Connected Network

- Used to solve the simple problem
- No backward connections



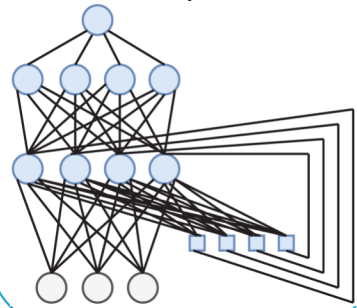
Convolutional Network

- Neurons in the same layer use identical weights (Weight sharing).
- Sparsely connected



Recurrent Neural Network (RNN)

- It has backward connections and contains cycles.



- More complex networks often consist of combinations of these three architectures.
- Networks are **often hybrids** of these three architectures.

EE708

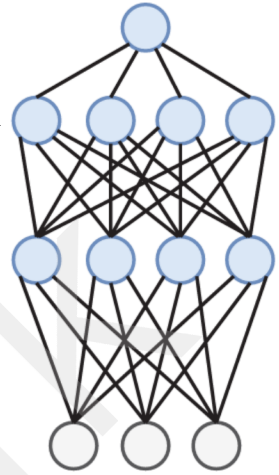
Module 6

40

Fully Connected Network

Fully Connected Network

- This type of network is used to solve for e.g. the XOR problem.
- There are **no backward connections** (also known as loops or cycles) in a feedforward network.



EE708

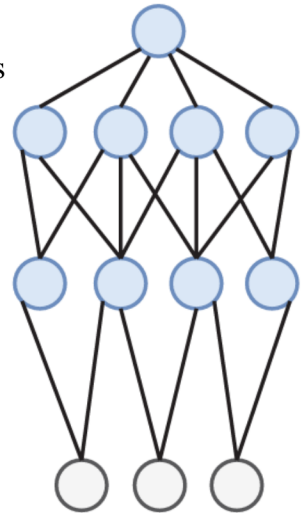
Module 6

41

CNN

Convolutional Network

- Individual neurons do not have their own unique weights; they use the same weights as other neurons in the same layer. This is a property known as weight sharing.
- Sparsely connected: considerably fewer connections than FCN.
- Excel in image classification problem.



EE708

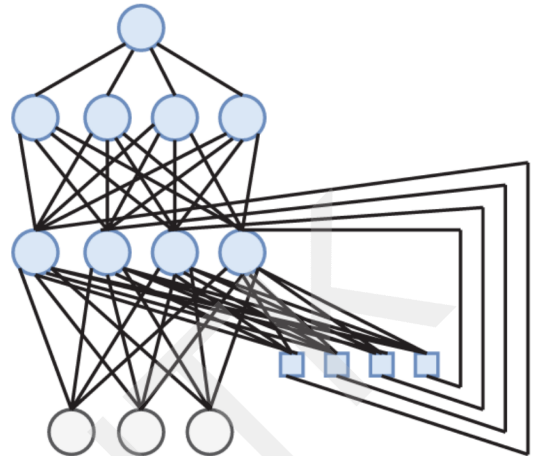
Module 6

42

RNN

Recurrent Neural Network (RNN)

As opposed to the feedforward network, the RNN has backward connections; that is, it is **not a directed acyclic graph (DAG)**, because it contains cycles.



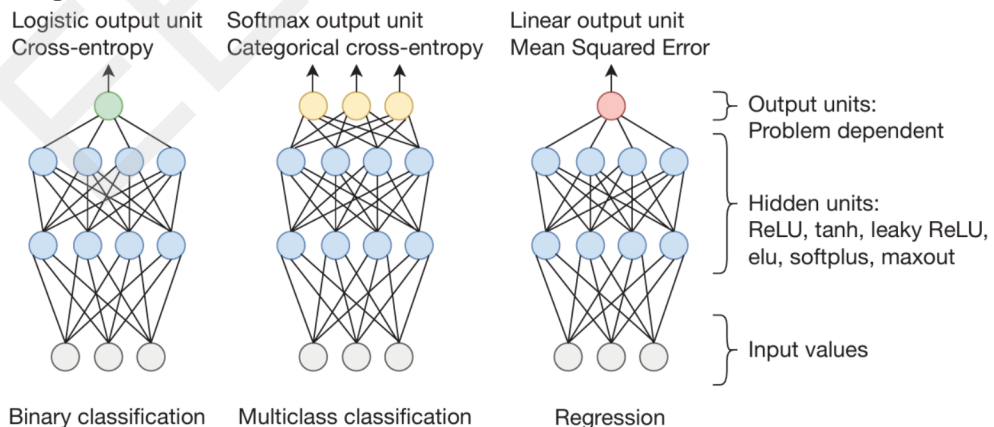
EE708

Module 6

43

Network Architecture : Selection of Output Units/Neurons

- The type of problem dictates the type of output unit and associated loss function.
- Multiple alternatives are available for the hidden layers.
- A good starting point is the rectified linear unit (ReLU). For some networks, other units result in better performance.



EE708

Types of units to use for different networks and layers.

44

Activation Functions

Logistic Sigmoid Function

Logistic sigmoid function: $S(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{e^z + 1}$

- Used for *binary classification problems*
- Output from a logistic sigmoid unit can be interpreted as a probability for a binary classification problem
- The recommended loss function to use with this type of output neuron is the *cross-entropy loss function*:

Cross entropy loss: $e(y) = -(y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}))$

where $\hat{y}$ is the output of the logistic sigmoid function and y is the desired output value.

EE708

Module 6

45

Activation Functions

Softmax

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

- Output from a softmax unit can be interpreted as a probability for a **multiclass classification problem**.
- The recommended loss function to use with this type of output neuron is the *cross-entropy loss function for multiclass classification*.

Cross entropy loss: $e(y) = - \sum_{c=1}^C y_c \ln(\hat{y}_c)$

EE708

Module 6

46

Activation Functions

Linear activation

- Used to predict a value not limited to the **range 0 to 1**.
- Uses the **identity function** as an activation function
- Does not have a problem with saturation, so **no need to use** the cross-entropy loss function.
- A good loss function is the **mean squared error (MSE)**.

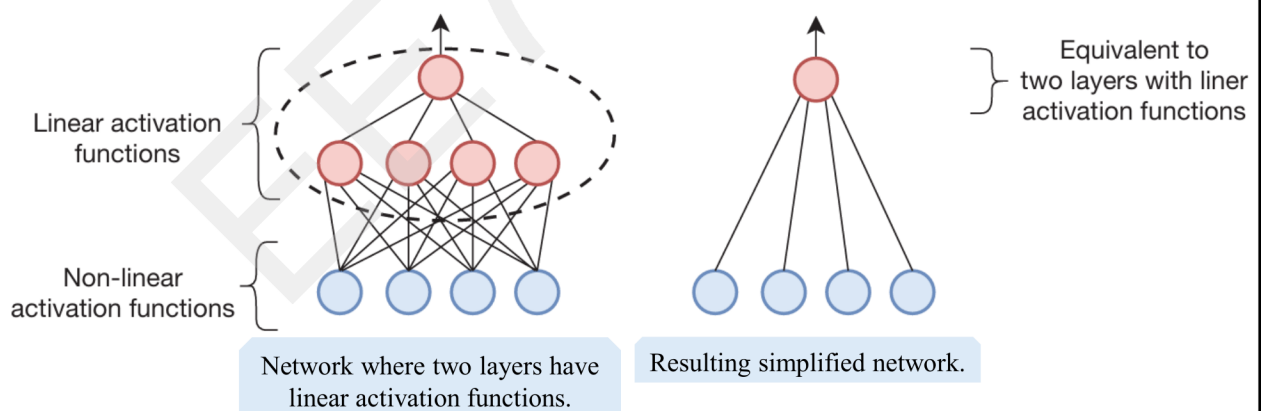
EE708

Module 6

47

Activation Functions : Combining Linear Activation Functions

Linear activation: Output = weighted sum (z)



**The two layers in the dashed oval collapses into a single layer.
Therefore, it is uncommon to stack linear layers.**

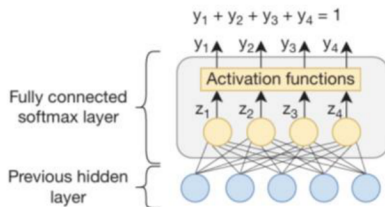
EE708

Module 6

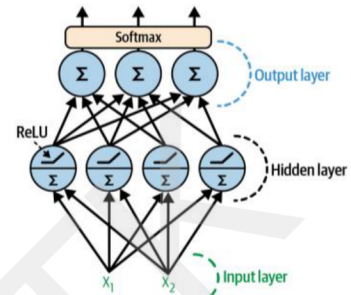
48

Activation Functions: Softmax for Multiclass Classification

- The softmax layer shown in the figure where the yellow circles compute only the weighted sum for each neuron, and the activation functions are applied in the yellow rectangle that has access to the logits (z-values) for all the neurons.



Fully connected softmax layer



A modern MLP (including ReLU and softmax) for classification.